REPUBLIQUE DU SENEGAL



Un peuple-un but-une foi

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Direction de l'Enseignement Supérieur Privé

Institut Supérieur d'Informatique

ISI

Examen: Prediction du diabète à l'aide du Machine Learning

Réalisé par:

Mme Bintou LO

Mme Badiayeu Saboudou SEYDI

Mme Ndeye Marieme DIALLO

M. Mahecor DIOUF

Encadré par:

M. BARRO

Année Académie : 2025 -2026

Table des matières

1. Introduction 4
3. Exploration et analyse des données 5
3.1 Dimensions du dataset 5
3.2 Informations sur les variables 5
3.4 Statistiques descriptives 7
3.5 Répartition de la variable cible 7
4. Préparation des données 9
4.1 Séparation des variables explicatives et de la cible 9
4.2 Séparation entraînement / test 9
4.3 Standardisation 10
5. Modèle 1 : Régression Logistique 11
5.1 Entraînement et prédiction 11
5.3 Matrice de confusion 12
5.4 Rapport de classification 13
6. Modèle 2 : Random Forest 14
6.1 Entraînement et prédiction 15
6.2 Matrice de confusion 16
7. Comparaison des modèles 18
7.1 Graphique de comparaison 19
9. Discussion des résultats 21
10. Conclusion 22

PLAN

1. Introduction
2. Présentation du jeu de données
3. Exploration et analyse des données
4. Préparation des données
5. Modèle 1: Régression Logistique
6. Modèle 2: Random Forest
7. Comparaison des modèles
8. Importance des variables
9. Discussion des résultats
10. Conclusion

1. Introduction

Le diabète est une maladie chronique qui constitue un problème important de santé publique. L'objectif de ce projet est d'utiliser les techniques de Machine Learning afin de construire un modèle capable de prédire si une personne est susceptible d'être diabétique à partir de plusieurs caractéristiques médicales.

Le travail a été réalisé avec Python dans Google Colab. Deux algorithmes de classification ont été étudiés: la Régression Logistique et Random Forest. Les performances des deux modèles ont ensuite été comparées afin d'identifier le modèle le plus performant sur les données de test.

2. Présentation du jeu de données

Le jeu de données utilisé contient 768 observations et 9 variables. Huit variables décrivent les caractéristiques des personnes et la neuvième, Outcome, correspond à la variable cible.

Variable	Description
Pregnancies	Nombre de grossesses
Glucose	Taux de glucose
BloodPressure	Pression artérielle
SkinThickness	Épaisseur de la peau
Insulin	Taux d'insuline
BMI	Indice de masse corporelle
DiabetesPedigreeFunction	Indicateur lié aux antécédents familiaux
Age	Âge
Outcome	Variable cible: 0 = non diabétique, 1 = diabétique

2.1 Chargement des données

```
import pandas as pd

df = pd.read_csv('/content/diabetes(1) (1).csv')

df.head()
```

Ce code importe la bibliothèque Pandas, charge le fichier CSV et affiche les cinq premières lignes du jeu de données grâce à la fonction df.head().

```
import pandas as pd

df = pd.read_csv('/content/diabetes(1) (1).csv')

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

3. Exploration et analyse des données

3.1 Dimensions du dataset

```
df.shape
```

```
df.shape

(768, 9)
```

Le résultat obtenu est (768, 9). Le dataset contient donc 768 lignes et 9 colonnes.

3.2 Informations sur les variables

```
df.info()
```

La commande df.info() permet de connaître le type des variables et le nombre de valeurs non nulles.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Indice de plage : 768 entrées, de 0 à 767
Colonnes de données (9 colonnes au total) :
 # Nombre de valeurs non nulles dans la colonne Dtype
--- ------ -------------- -----
 0 grossesses 768 non-null int64
 1 Glucose 768 int64 non nul
 2 Pression artérielle 768 entier non nul 64 bits
 3 Épaisseur de peau 768 entier non nul 64 bits
 4 Insuline 768 non-null int64
 5 IMC 768 non nul flottant64
 6 DiabetesPedigreeFunction 768 non-null float64
 7 Âge 768 non nul int64
 8 Résultat 768 int64 non nul
dtypes : float64(2), int64(7)
Utilisation de la mémoire : 54,1 Ko
```

Les 9 colonnes comportent chacune 768 valeurs non nulles. **3.3**

Vérification des valeurs nulles et manquantes

df.isnull().sum()

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Cette vérification permet de compter les valeurs nulles (NaN) dans chaque colonne. Dans notre dataset, le résultat est 0 pour toutes les variables : il n'y a donc aucune valeur manquante détectée par Pandas.

NB : Il faut distinguer les valeurs nulles des valeurs égales à zéro. Certaines variables médicales contiennent des zéros qui peuvent être des valeurs réelles ou des mesures non renseignées selon la variable.

3.4 Statistiques descriptives

df.describe()

La fonction describe() fournit des statistiques descriptives telles que la moyenne, l'écart-type, le minimum, le maximum et les quartiles des variables numériques.

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

3.5 Répartition de la variable cible

df['Outcome'].value_counts()

```
df['Outcome'].value_counts()
```

```
...
```

Outcome	count
0	500
1	268

```
dtype: int64
```

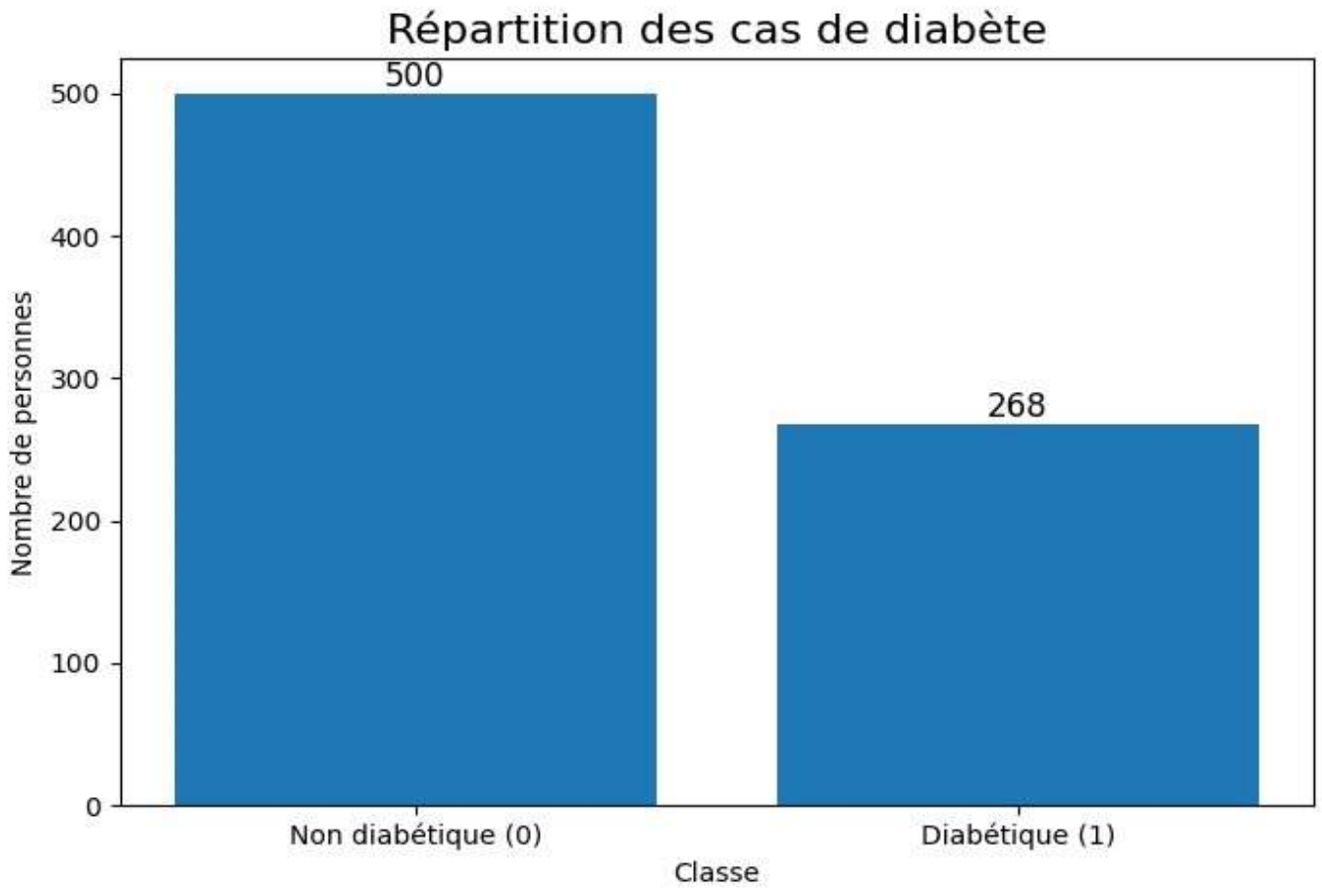
La variable Outcome est la variable cible. La valeur 0 représente les personnes non diabétiques et la valeur 1 représente les personnes diabétiques. Le dataset contient 500 observations de classe 0 et 268 observations de classe 1.

```
import matplotlib.pyplot as plt

df['Outcome'].value_counts().plot(kind='bar')

plt.title('Répartition des cas de diabète')
plt.xlabel('Outcome (0 = Non diabétique, 1 = Diabétique)')
plt.ylabel('Nombre de personnes')
plt.show()
```

Ce graphique permet de visualiser la répartition des deux classes.



[8]

4. Préparation des données

4.1 Séparation des variables explicatives et de la cible

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

print("X :", X.shape)
print("y :", y.shape)
```

X contient les huit variables utilisées pour effectuer les prédictions, tandis que y contient la variable cible Outcome.

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

print("X :", X.shape)
print("y :", y.shape)

X : (768, 8)
y : (768,)
```

X possède 768 observations et 8 variables, alors que y contient 768 valeurs.

4.2 Séparation entraînement / test

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=42,
```

Les données ont été divisées en 80 % pour l'entraînement et 20 % pour le test.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Données d'entrainement :", X_train.shape)
print("Données de test :", X_test.shape)

Données d'entrainement : (614, 8)
Données de test : (154, 8)
```

On obtient 614 observations pour l'entraînement et 154 observations pour le test.

4.3 Standardisation

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Standardisation terminée !")
```

La standardisation permet de mettre les variables numériques sur une échelle comparable. Le scaler est ajusté uniquement sur les données d'entraînement puis appliqué aux données de test.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Standardisation terminée !")

Standardisation terminée !
```

5. Modèle 1 : Régression Logistique

La Régression Logistique est un algorithme de classification adapté à un problème comportant deux classes. Dans notre cas, le modèle doit prédire 0 (non diabétique) ou 1 (diabétique).

5.1 Entraînement et prédiction

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

La méthode fit() entraîne le modèle sur les données d'entraînement. La méthode predict() permet ensuite d'obtenir les prédictions sur les données de test.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)

print("Modèle entraîné !")
```

```
Modèle entraîné !
```

```
y_pred = model.predict(X_test)

print(y_pred)
```

```
[1 0 0 0 0 0 1 0 1 0 1 0 0 0 0 1 0 1 0 0 1 0 1 1 0 1 0 0 0 0 0 0 1 1 0 0
 0 1 1 0 0 0 0 0 0 0 0 1 1 1 1 0 0 1 0 1 0 1 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 0 1 1 0 0 0 0 0 1 0 1 0 1 0 0
 1 0 0 0 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1
 1 0 0 0 1 0]
```

5.2 Accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", round(accuracy * 100, 2), "%")
```

Accuracy = 71,43 %.

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", round(accuracy * 100, 2), "%")
```

```
Accuracy : 71.43 %
```

5.3 Matrice de confusion

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[82 18]
 [26 28]]
```

Le modèle a correctement classé 82 personnes non diabétiques et 28 personnes diabétiques. Il a commis 18 faux positifs et 26 faux négatifs.

```
import matplotlib.pyplot as plt

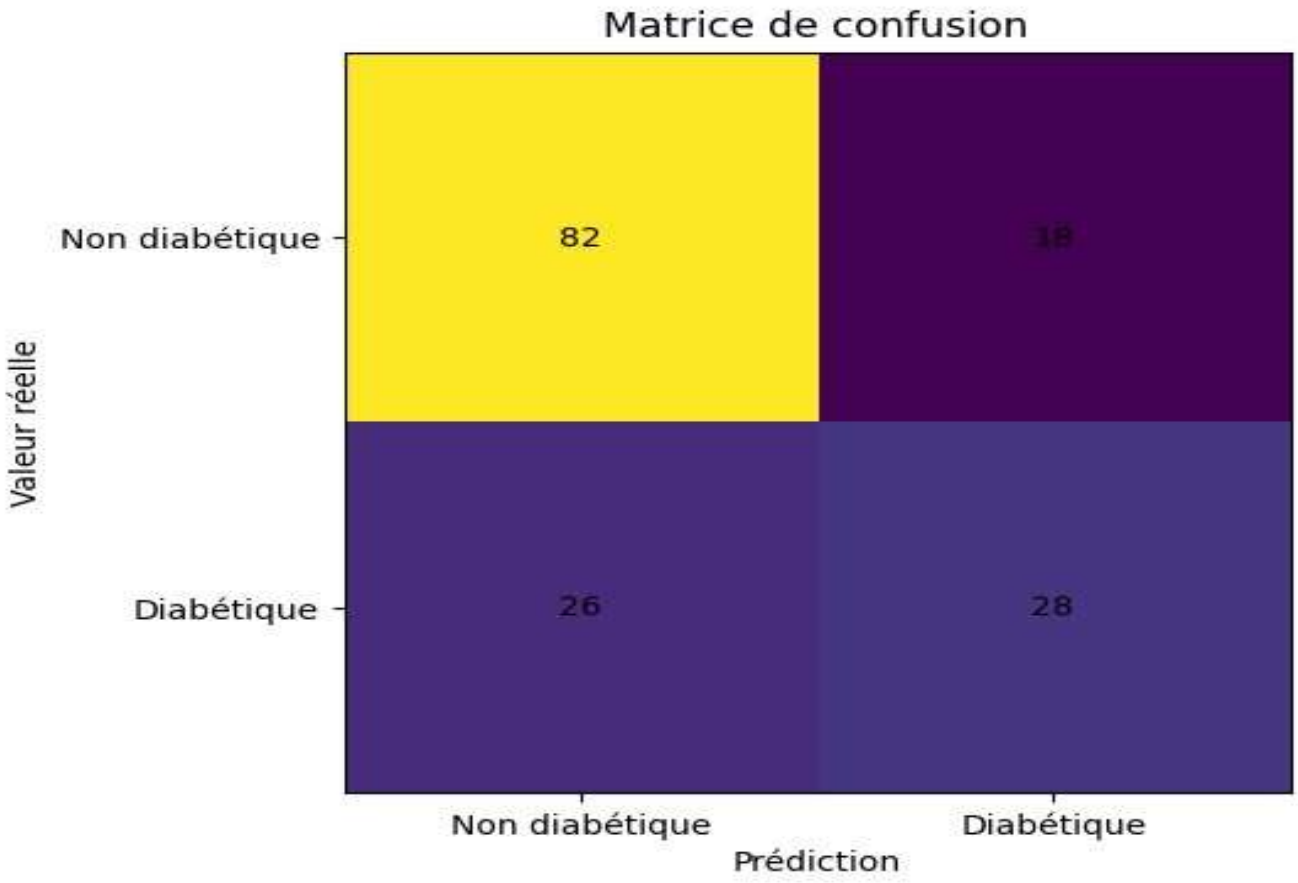
plt.figure(figsize=(6, 5))
plt.imshow(cm)

plt.title('Matrice de confusion')
plt.xlabel('Prédiction')
plt.ylabel('Valeur réelle')

plt.xticks([0, 1], ['Non diabétique', 'Diabétique'])
plt.yticks([0, 1], ['Non diabétique', 'Diabétique'])

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.show()
```



5.4 Rapport de classification

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Classe	Precision	Recall	F1-score
0 - Non diabétique	0,76	0,82	0,79
1 - Diabétique	0,61	0,52	0,56

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.76      0.82      0.79       100
           1       0.61      0.52      0.56        54

    accuracy                           0.71       154
   macro avg       0.68      0.67      0.67       154
weighted avg       0.71      0.71      0.71       154
```

Pour la classe 1 (diabétique), la précision est de 0,61, le rappel de 0,52 et le F1-score de 0,56. Le modèle reconnaît donc moins bien la classe diabétique que la classe non diabétique.

6. Modèle 2 : Random Forest

Random Forest est un algorithme qui combine plusieurs arbres de décision. Il peut modéliser des relations plus complexes entre les variables et permet également d'obtenir une estimation de l'importance des variables.

6.1 Entraînement et prédiction

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

accuracy_rf = accuracy_score(y_test, y_pred_rf)

print("Accuracy Random Forest :", round(accuracy_rf * 100, 2), "%")
```

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

accuracy_rf = accuracy_score(y_test, y_pred_rf)

print("Accuracy Random Forest :", round(accuracy_rf * 100, 2), "%")
```

```
Accuracy Random Forest : 75.97 %
```

Le modèle Random Forest utilisé contient 100 arbres. Le résultat obtenu est une accuracy de 75,97 %.

6.2 Matrice de confusion

cm_rf = confusion_matrix(y_test, y_pred_rf)

print(cm_rf)

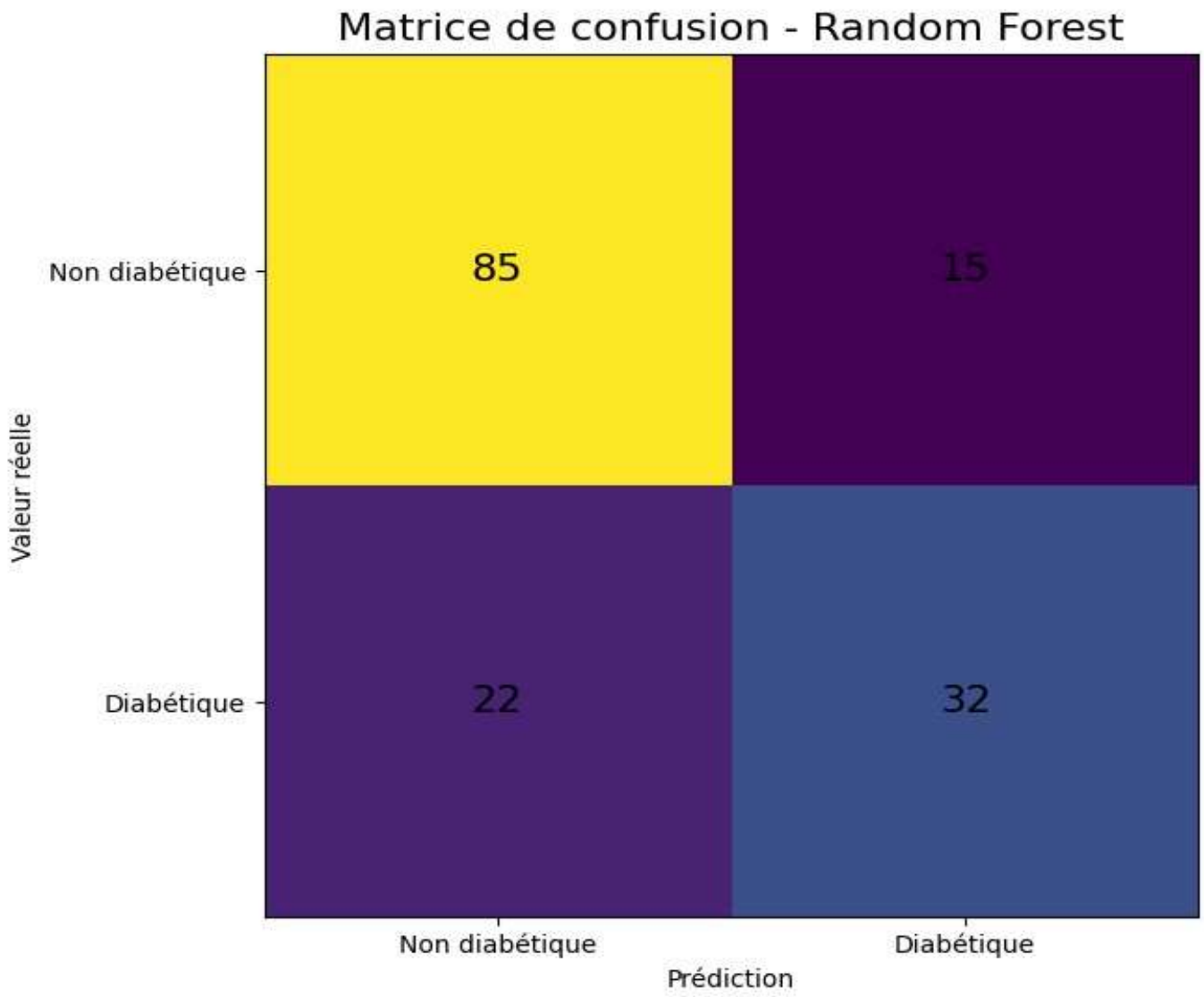
```
from sklearn.metrics import confusion_matrix

cm_rf = confusion_matrix(y_test, y_pred_rf)

print(cm_rf)
```

```
[[85 15]
 [22 32]]
```

Random Forest a correctement classé 85 personnes non diabétiques et 32 personnes diabétiques. Il a produit 15 faux positifs et 22 faux négatifs.



6.3 Rapport de classification

print(classification_report(y_test, y_pred_rf))

	lasse	Precision	Recall	F1-score	
0	- Non diabétique	0,79	0,85	0,82	
1	- Diabétique	0,68	0,59	0,63	

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.79      0.85      0.82       100
           1       0.68      0.59      0.63        54

    accuracy                           0.76       154
   macro avg       0.74      0.72      0.73       154
weighted avg       0.75      0.76      0.76       154
```

Pour la classe 1, le F1-score augmente à 0,63 et le recall à 0,59. Random Forest améliore donc la détection de la classe diabétique par rapport à la Régression Logistique.

7. Comparaison des modèles

Modèle	Accuracy	F1 classe 1	Faux négatifs
Régression Logistique	71,43 %	0,56	26
Random Forest	75,97 %	0,63	22

Random Forest obtient la meilleure accuracy (75,97 % contre 71,43 %) et un meilleur F1-score pour la classe diabétique (0,63 contre 0,56). Il produit également moins de faux négatifs : 22 contre 26.

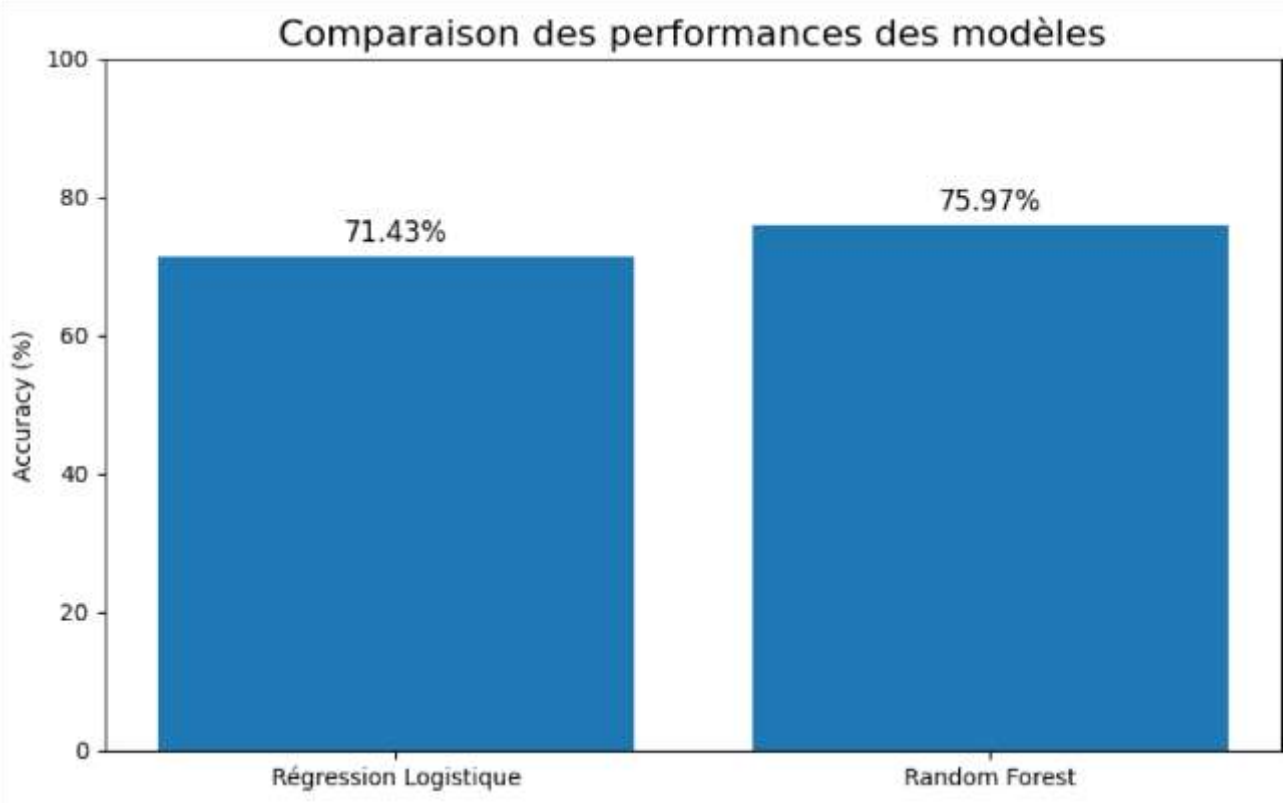
```
models = ['Régression Logistique', 'Random Forest']
accuracies = [71.43, 75.97]

plt.figure(figsize=(8, 5))
bars = plt.bar(models, accuracies)

plt.title('Comparaison des performances des modèles')
plt.ylabel('Accuracy (%)')
plt.ylim(0, 100)

for bar, value in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width()/2,
        value + 2,
        f'{value:.2f}%',
        ha='center'
```

7.1 Graphique de comparaison



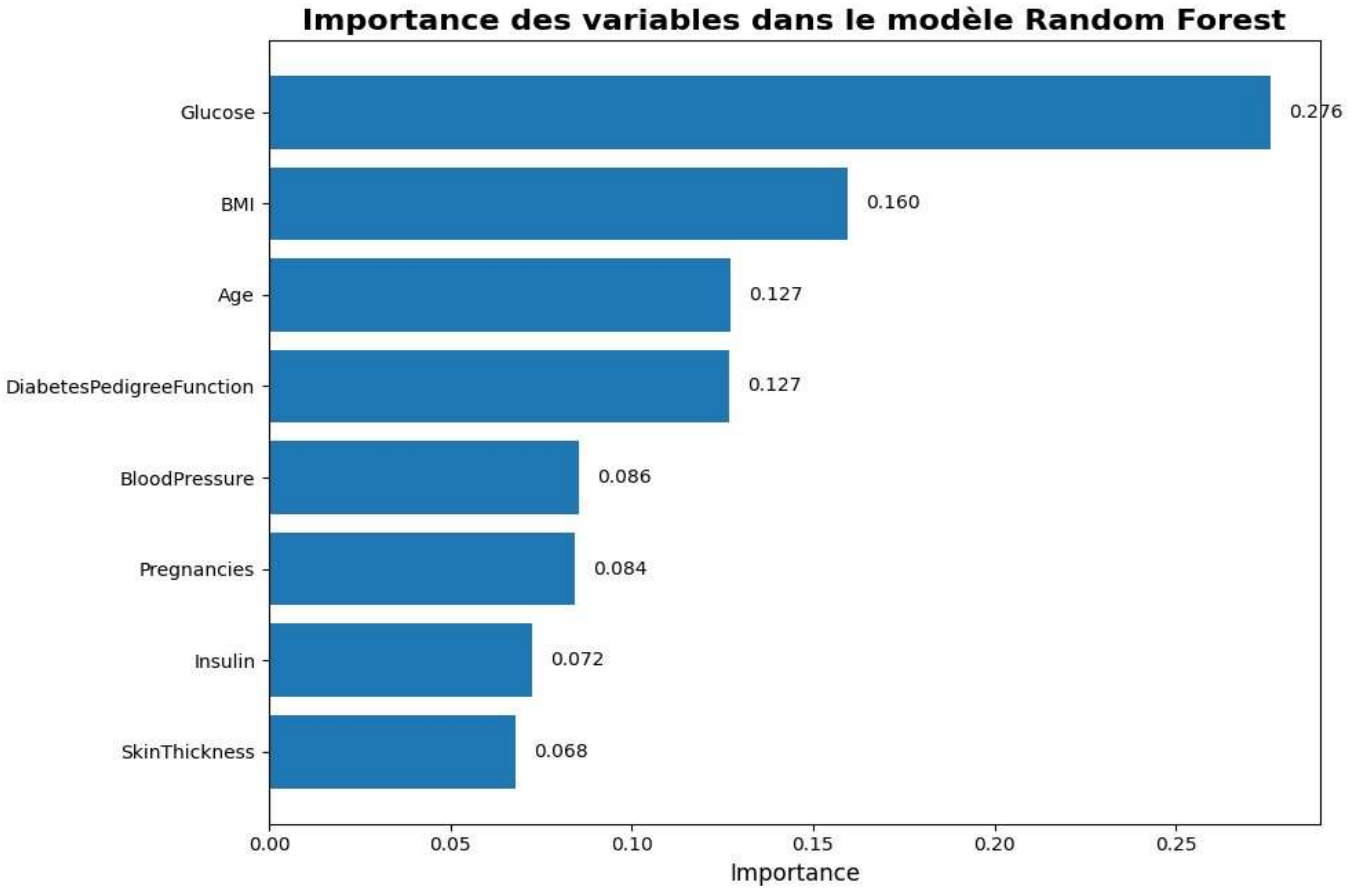
8. Importance des variables

```
importance = pd.Series(
rf_model.feature_importances_,
index=df.drop('Outcome', axis=1).columns
).sort_values(ascending=True)

plt.figure(figsize=(10, 7))
bars = plt.barh(importance.index, importance.values)

plt.title(
'Importance des variables dans le modèle Random Forest'
)
plt.xlabel('Importance')

for bar, value in zip(bars, importance.values):
plt.text(
value + 0.005,
bar.get_y() + bar.get_height()/2,
```



Le graphique montre l'importance relative des variables selon Random Forest. Dans notre modèle, Glucose est la variable la plus importante, suivie notamment par BMI, Age et DiabetesPedigreeFunction. Cette importance indique la contribution des variables aux décisions du modèle et ne doit pas être interprétée comme une preuve de causalité.

9. Discussion des résultats

Les deux modèles testés permettent de prédire la variable Outcome, mais Random Forest obtient de meilleurs résultats sur l'ensemble de test. Son accuracy atteint 75,97 %, soit 4,54 points de pourcentage de plus que la Régression Logistique.

La matrice de confusion de Random Forest montre 32 vrais positifs et 22 faux négatifs, contre 28 vrais positifs et 26 faux négatifs pour la Régression Logistique. Le modèle Random Forest semble donc mieux détecter les cas positifs dans cette expérimentation,

même si son recall de 0,59 montre qu'une partie des personnes diabétiques n'est toujours pas correctement détectée.

10. Conclusion

Ce projet a permis de mettre en pratique les principales étapes d'un projet de Machine Learning : chargement des données, exploration, préparation, séparation entre entraînement et test, standardisation, entraînement de modèles, prédiction et évaluation.

Deux modèles de classification ont été étudiés : la Régression Logistique et Random Forest. La Régression Logistique a obtenu une accuracy de 71,43 %, tandis que Random Forest a atteint 75,97 %. Dans le cadre de cette expérimentation, Random Forest est donc le modèle le plus performant des deux.

Cependant, une accuracy de 75,97 % signifie que le modèle n'est pas parfait. Les résultats montrent notamment que certains cas diabétiques sont encore mal classés. Le modèle doit donc être considéré comme un exercice de Machine Learning et non comme un outil de diagnostic médical.